Experiment No. 3

Edge Computing Lab

Name of Student:	
Class :	**Date of Performance:**
Roll No :	**Date of Checking :**
	Sign :

Containerization & Orchestration on Raspberry Pi via a REST API

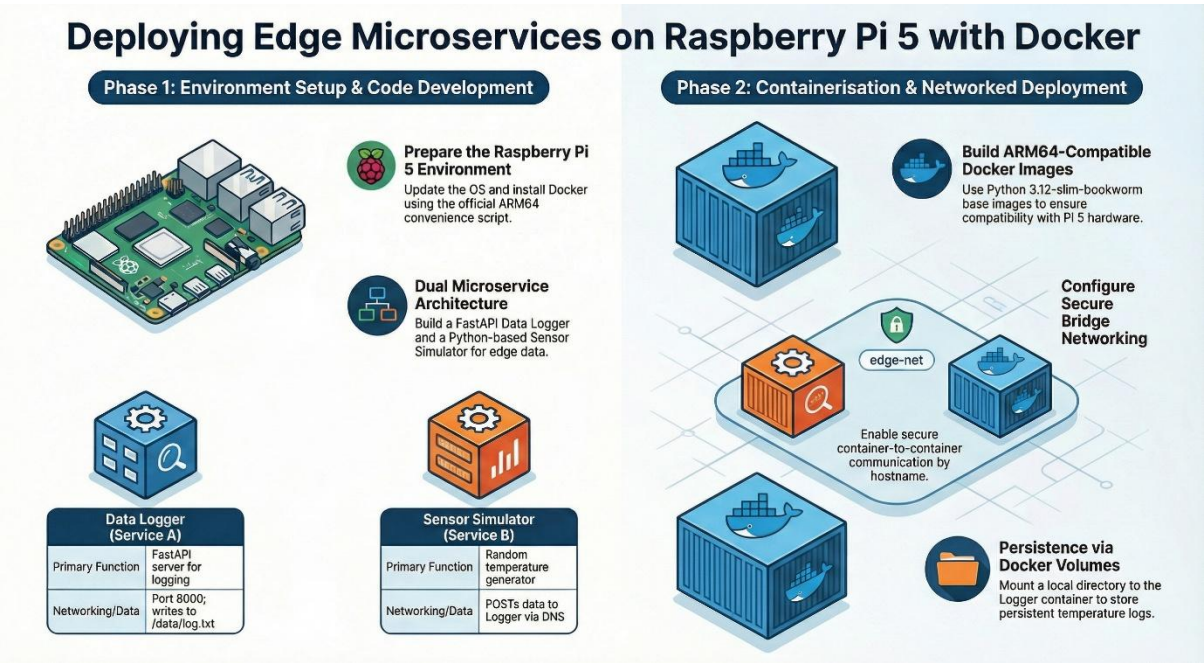
Objective: To demonstrate the fundamentals of Edge Computing by installing Docker on a Raspberry Pi and deploying a multi-container architecture where two independent microservices communicate via a REST API

Tasks:

1. **Environment Setup:**
   - Update the Raspberry Pi OS and install the Docker Engine.
   - Verify the installation by running the hello-world container.
2. **Microservice Development:**
   - Service A (Data Logger): Develop a Python script using Flask/FastAPI that acts as a server to receive and save data to a local file.
   - Service B (Sensor Simulator): Develop a separate Python script that generates dummy sensor values (e.g., random temperature data) and sends them to Service A via HTTP POST requests.
3. **Containerization:**
   - Write a unique Dockerfile for each service (Service A and Service B).
   - Build the Docker images for both services directly on the Raspberry Pi to ensure architecture compatibility (ARM64).
4. **Deployment & Networking:**
   - Create a Docker Bridge Network to allow the containers to communicate securely.

Run both containers on this network and verify that the "Sensor Simulator" is successfully sending data to the "Data Logger."

Block Diagram



Step 1: Nuclear Reset

Before we start fresh, we must kill any "zombie" containers, networks, or old files that are causing the Conflict errors.

Bash

Stop and remove all containers

```
docker stop data-logger sensor-sim 2>/dev/null
docker rm data-logger sensor-sim 2>/dev/null
```

Remove the network

```
docker network rm sensor-net 2>/dev/null
```

Clear folder in our Main EC Labs Folder to start fresh

mkdir Expt3

Step 2: Prepare the Correct Files

Use nano or your preferred editor to ensure the files contain **exactly** this code. I have added the encoding fix and the unbuffered logging flag.

File 1: requirements.txt

```Plaintext
fastapi
uvicorn
pydantic
requests
```

File 2: data_logger.py

Python

```
from fastapi import FastAPI
from pydantic import BaseModel
import os

app = FastAPI()
log_file = "/data/log.txt"

class SensorData(BaseModel):
    temperature: float

@app.post("/log")
async def log_data(data: SensorData):
    # Get the date from the system
    timestamp = os.popen('date').read().strip()
    with open(log_file, "a") as f:
        f.write(f"Temperature: {data.temperature}C: at {timestamp}\n")
    return {"status": "logged"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

File 3: sensor_sim.py

Python

```python
# -*- coding: utf-8 -*-
import requests
import random
import time
import os

# Use the container name as the hostname
SERVICE_A_URL = os.getenv("SERVICE_A_URL", "http://data-logger:8000/log")

print("Simulator started. Sending data every 10 seconds...")

while True:
    temp = round(random.uniform(20.0, 30.0), 2)
    try:
        response = requests.post(SERVICE_A_URL, json={"temperature": temp})
        print(f"Sent temp {temp}C: Status {response.status_code}")
    except Exception as e:
        print(f"Connection Error: {e}")
    time.sleep(10)
```

Step 3: Create the Dockerfiles

We will use python:3.11-slim to match your requirements.

Dockerfile.logger

```Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY data_logger.py .
RUN mkdir -p /data
# ENV ensures logs appear in 'docker logs' immediately
ENV PYTHONUNBUFFERED=1
CMD ["uvicorn", "data_logger:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile.simulator

```Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY sensor_sim.py .
ENV PYTHONUNBUFFERED=1
CMD ["python", "sensor_sim.py"]
```

Step 4: Build and Execute (The "Accurate Way")

1. **Create the Bridge Network:**

```
docker network create sensor-net
```

2. **Build the Images:**

```
docker build -f Dockerfile.logger -t data-logger .
docker build -f Dockerfile.simulator -t sensor-sim .
```

3. **Run Service A (Data Logger):**

```
docker run -d \
  --name data-logger \
  --network sensor-net \
  -v $(pwd)/data:/data \
  data-logger
```

4. **Run Service B (Sensor Simulator):**

```
docker run -d \
  --name sensor-sim \
  --network sensor-net \
  -e SERVICE_A_URL=http://data-logger:8000/log \
  sensor-sim
```

Step 5: Verify the Output

Since we added PYTHONUNBUFFERED=1, the logs will no longer be blank.

Check the Simulator's activity:

```
docker logs -f sensor-sim
```

You should see: Sent temp 25.43C: Status 200

Check the saved data on your Raspberry Pi:

```
cat data/log.txt
```

You should see: Temperature: 25.43C: at [Date/Time]

1. The Component Relationships

Every file you created serves a specific role in the "Container Lifecycle."

- **Python Scripts (.py)**: These are your **Applications**. They contain the "business logic" (generating data or saving data). They are unaware of Docker; they only care about Python and the network URL they are told to talk to.
- **Dockerfiles**: These are the **Blueprints**. They tell Docker how to build an "Environment" for the Python scripts. They ensure that even though you are on a Raspberry Pi (ARM architecture), the container has exactly Python 3.11.2 and the libraries listed in requirements.txt.
- **Requirements.txt**: This is the **Dependency List**. It ensures both containers have the same versions of requests or fastapi, preventing "it works on my machine" errors.
- **Docker Bridge Network**: This is the **Virtual Switch**. In Docker, containers are isolated. The network sensor-net acts like a private Wi-Fi router that only these two containers are connected to. It provides **Service Discovery**, allowing the Simulator to find the Logger by its name (data-logger) instead of a changing IP address.

---

2. The Data Flow (Step-by-Step)

1. **Generation**: Inside sensor-sim, the script creates a random float.

2. **Serialization**: The requests library takes that float and turns it into a **JSON** object: {"temperature": 25.5}.
3. **Transmission**: The Simulator sends an **HTTP POST** request. It asks the Docker Network: *"Where is 'data-logger'?"* The network resolves that name to the Logger's internal IP.
4. **Reception**: The data-logger (running FastAPI) sits in a "listening" state on port 8000. It receives the JSON, validates it using the SensorData model, and extracts the number.
5. **Persistence (The Volume)**: The Logger writes the data to /data/log.txt. Because of the **Volume Mapping** (-v $(pwd)/data:/data), Docker redirects that write operation from the container's temporary memory to your Raspberry Pi's physical SD card.

---

3. Theory: Why use Docker for this?

In a real-world Edge AI project—like your work with **Seizure Prediction** or **REM sleep monitoring** —you wouldn't want one giant program.

- **Isolation**: If the Sensor Simulator crashes because a physical sensor is unplugged, the Data Logger stays alive, keeping your previous data safe.
- **Scalability**: You could eventually run five different "Simulator" containers (representing five different rooms/patients), and they could all send data to the single "Logger" container without changing any code.
- **Portability**: Since you built this on a Raspberry Pi 5, you could move these same Docker images to a Jetson Nano or a Cloud Server, and they would behave exactly the same way.

4. Architectural Diagram

Summary Table of the Flow

Phase	Component	Action
Trigger	sensor_sim.py	Generates a random temperature every 10 seconds.
Transport	Docker Network	Routes the HTTP POST request using the hostname data-logger.
Processing	data_logger.py	FastAPI validates the JSON and attaches a system timestamp.
Storage	Docker Volume	Bridges the container's /data folder to the Pi's ./data directory.